

```

int RecursiveRepetition (int v[], int tam) {
    if (tam <= 0) return 0;
    int j = 1;
    for (int i = 1; i < tam; i++) {
        if (v[i] != v[i-1]) v[j++] = v[i];
    }
    return j;
}

```

```

void ordena (int v[], int tam) {
    int i, j, aux;
    for (i = 0; i < tam - 1; i++) {
        for (j = 0; j < tam - 1 - i; j++) {
            if (v[j] > v[j+1]) {
                aux = v[j];
                v[j] = v[j+1];
                v[j+1] = aux;
            }
        }
    }
}

```

```
void pseudoPrime(int v[], int tam) {
```

```
    int num = 2;
```

```
    int count = 0;
```

```
    while(count < tam) {
```

```
        if(ehPrime(num)) {
```

```
            v[count] = num
```

```
            count++;
```

```
        }  
        num++;
```

```
    }
```

```
}
```

```
int ehPrime(int h) {
```

```
    if(h < 2) return 0;
```

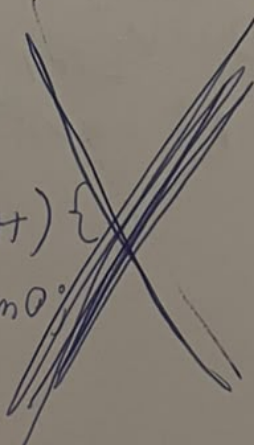
```
    for(int i = 2; i * i <= h; i++) {
```

```
        if(h % i == 0) return 0;
```

```
    }
```

```
    return 1;
```

```
}
```



```
Void mainPerLink(int m[] Row Columns, int lin, int col, int v[]) {  
    for(int i=0; i < lin; i++){  
        v[i] = m[i][0];
```

```
        for(int j=1; j < col; j++){  
            if(m[i][j] > v[i]){  
                v[i] = m[i][j];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
void invertirPalabra(char str[]) {
```

```
    int inicio = 0; int
```

```
    for (int i = 0; i++) {
```

```
        if (str[i] == ' ' || str[i] == '\0') {
```

```
            int fin = i - 1;
```

```
            while (inicio < fin) {
```

```
                char aux = str[inicio];
```

```
                str[inicio] = str[fin];
```

```
                inicio++;
```

```
                fin--;
```

```
            }
```

```
            inicio = i + 1;
```

```
        } if (str[i] == '\0') break;
```

```
    }
```

```
}
```